



UNIVERSITÀ DI PISA

Computer Engineering
Distributed Systems

Distributed Crazy Time

Gabriele Caioli
Klaudio Ciacia
Lorenzo Vanni

Academic Year: 2025/2026

1. Introduction	3
1.1 Environment	3
2. System Description	4
2.1 Use Cases	4
2.2 The game	4
2.2.1 Minigames	5
2.3 Architecture	5
2.2.1 Central node (Java Gateway)	7
2.2.2 Message Broker (RabbitMQ)	7
2.2.3 Game Server Cluster (Erlang Engine)	7
2.4 Dealer Election	7
2.5 Synchronization	8
2.5.1 Snapshots	8
2.5.2 State recovery	8
3. Implementation	8
3.1 Database Node	8
3.1.1 User table	8
3.1.2 Bet table	9
3.2 Central Node (Java)	9
3.2.1 Source code structure	9
3.2.2 The config package	9
3.2.3 The controller package	9
3.2.4 The rabbitmq package	10
3.3 Erlang Game Engine	10
3.3.1 Source code structure	10
3.3.2 Fault Tolerance Modules	10
3.3.3 Core Game Modules	10
3.3.4 Mini-games	11
4. Demonstration	11
4.1 Run the system	11
4.2 Interaction	12

1. Introduction

This project consists of a replica of the casino game Crazy Time, implemented as a web based application with three main layers:

- A Web Layer (Frontend in HTML/CSS/JS and a Gateway implemented in Java Spring Boot) responsible for user management, wallets, and HTTP API serving.
- A Message Broker Layer using RabbitMQ for asynchronous communication between the Gateway and the Game Engine.
- A Distributed Game Engine implemented as a cluster of Erlang nodes, responsible for managing the state of the game, fair random number generation, executing the main wheel spins, and handling the interactions of the 4 asynchronous mini-games.

This architecture allows the system to guarantee real-time responsiveness and strong fault tolerance.



1.1 Environment

The Java-based part of the system (the gateway) has been implemented using Spring Boot, interacting with a local database for persistence. The Erlang-based components (the game engine cluster) have been developed using Erlang/OTP, utilizing Mnesia as a replicated distributed database to store the state. The communication middleware relies on RabbitMQ (AMQP Protocol).

2. System Description

The system is a web-based application providing functionalities for both account and wallet management, and real-time gameplay. The main supported operations include user registration, checking the wallet balance, placing bets on the main wheel or mini-games segments, and interacting during the mini-games phases.

2.1 Use Cases

An Unregistered User can:

- Register to the service to create a wallet.

An Unlogged User can:

- Login to the service as a Player.

A Logged Player can:

- Logout;
- View the live countdown timer and the current state of the wheel/mini-games;
- Place bets on one or more wheel segments (if the betting window is open);
- View their current wallet balance and betting history.

The System must:

- Maintain and update the Players' wallet balances securely;
- Synchronize the countdown timer and game status across all connected clients;
- Reject any bets placed after the "No more bets" signal;
- Generate a single, authoritative random outcome for each wheel spin and mini-game event;
- Gracefully recover from the crash of the node generating the game outcomes, ensuring the game continues without state corruption.

2.2 The game

Crazy Time is a live casino game with the following phases in each round:

1. Betting: the user can bet some amount of money on any outcome of the wheel spin (numbers or minigames). This lasts until a timer expires.
2. Lock: the bets are locked in and no more bets are accepted.
3. Spin: the wheel spins and an outcome is decided based on the result
4. Resolution: if the wheel lands on a number space, players who bet on that number receive a payout of the original bet multiplied by the number; if the wheel lands on a minigame space an interactive minigame starts for players who bet on that space; the bets on the space the wheel lands on are refunded and the other ones are lost
5. Minigame (optional): if the wheel lands on a minigame space, the minigame starts; there are four possible minigames, two of which have an interactive element (crazy time and cash hunt) and two that do not (pachinko and coin flip)

Clients do not communicate directly with Erlang; they communicate with the Java Gateway. The Java Gateway translates HTTP requests into AMQP messages.

- Betting: Users place bets via `/api/wallet/place-bet`. The backend verifies the balance and issues a message to RabbitMQ.

- Mini-game interaction: During mini-games, users submit their choices (e.g., choosing a specific arrow in CrazyTime) to the Java API, which forwards them to the engine.

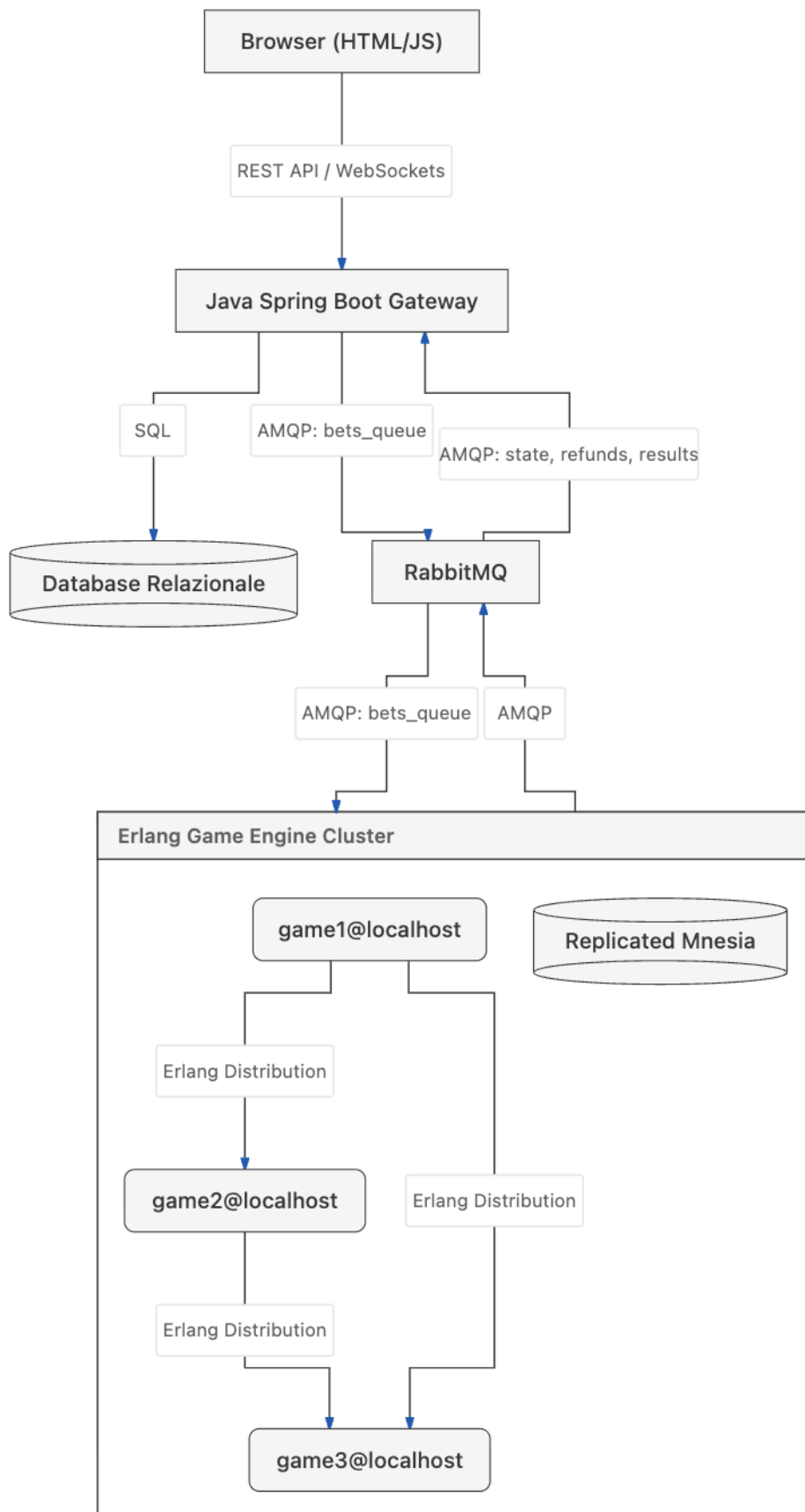
2.2.1 Minigames

There are four possible minigames that can start after a wheel spin:

- Coin flip: two random multipliers are chosen and assigned to the two sides of a coin, then the coin is flipped and the players who bet on the minigame receive a payout of their bet multiplied by the multiplier decided by the outcome of the flip
- Pachinko: a pachinko machine appears and random multipliers are chosen for the bottom slots the ball could land in, then a ball is dropped in the machine from the top in a random spot and players receive a payout of their bet multiplied by the number the ball landed on
- Cash hunt: a grid of 108 multipliers is shown to the players, then each one is quickly covered by a random symbol from a set of 24 and shuffled around the board; players have a set amount of time to choose a spot on the board; the multipliers are revealed and each player gets a payout of their bet multiplied by the number under the symbol they chose
- Crazy time: a second wheel appears, with three arrows at the top instead of one and various multipliers on the spaces, players have time to choose one of the selectors before the spin, then the wheel spins and each player gets a payout of their bet multiplied by the number that landed on the arrow they chose

2.3 Architecture

The architecture is designed to decouple the client-facing APIs from the authoritative game logic, using a message queue.



2.2.1 Central node (Java Gateway)

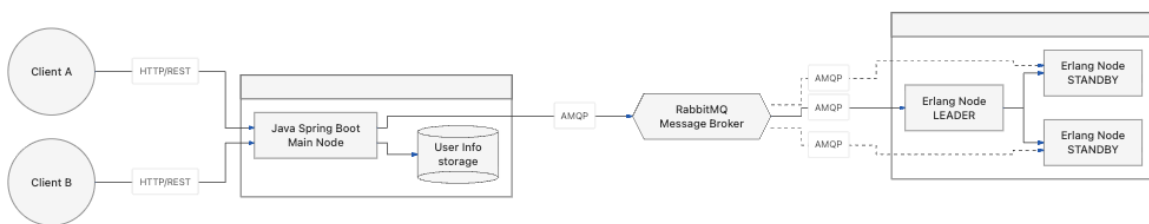
The central node is implemented using the Spring Boot framework. It provides the REST APIs for user registration, wallet management, and placing bets. It is the only component that interacts directly with the relational database holding user accounts and balances. When a bet is placed, the Gateway deducts the balance locally and forwards a structured JSON message containing a unique `bet_id` via RabbitMQ to the Erlang Engine. It also listens to specific queues to process payouts or refunds sent back by the engine.

2.2.2 Message Broker (RabbitMQ)

Native AMQP integration allows full decoupling. The Gateway produces to a `bets_queue` consumed by the Erlang cluster, and listens to queues for state updates (`state_queue`), game results (`results_queue`), and refunds (`refunds_queue`).

2.2.3 Game Server Cluster (Erlang Engine)

The Erlang game engine runs on a cluster of nodes (typically 3). The game server acts authoritatively: it decides when the betting window closes, calculates the global ledger of accepted bets, spins the wheel, and resolves the outcomes. The nodes discover each other and form a highly available cluster.



2.4 Dealer Election

The Erlang game engine runs on multiple nodes at once, if they all processed the game independently they would all spin their own wheel and pay out winnings to customers, who might even received duplicated payouts. To solve this issue a Dealer (Leader) node is elected, and it will be the only one who spins the wheel and calculates payouts. The election is done using the Bully algorithm:

1. Each node has a unique identifier (e.g., `game1`, `game2`, `game3`).
2. The node with the highest ID always wins the election and acts as the Leader.
3. The other nodes act in a Standby role, simply forwarding incoming bets to the Leader.
4. The system requires a Quorum (majority of nodes) to operate, preventing Split-Brain scenarios in case of network partitions.
5. If the current Leader crashes, the remaining nodes detect the failure and immediately hold a new election. The highest ID among the survivors becomes the new Leader.

2.5 Synchronization

2.5.1 Snapshots

Users playing the game only have a certain amount of time to place bets, after which bets are locked in and the game starts for everyone, so the system needs to know which bets were made in time and which were made too late and need to be rejected. Since Erlang processes and AMQP queues operate asynchronously, the **Chandy-Lamport Snapshot Algorithm** has been implemented to avoid race conditions and decide accurately the cutoff point for valid bets:

1. **Competing Consumers:** Workers on all nodes consume bets from RabbitMQ asynchronously and forward them to the Leader's `wheel_process`.
2. **The Marker:** When the timer expires, the `wheel_process` (Leader) saves its local state (received bets and extracted winner) and sends a "Snapshot Marker" backwards to all workers.
3. **In-Flight Messages:** Workers receive the Marker, record their local state, and send a Marker back to the Leader.
4. **Capturing Channel State:** Crucially, any bet that arrives at the Leader after the Leader started the snapshot, but before it receives the Marker back from the worker, was effectively "in-flight" across the network when the gong rang. These in-flight bets are causally part of the current round and are rightfully captured and included.
5. **The Ledger:** A dedicated Collector process aggregates these local and channel states to form the authoritative Ledger of the round, which is saved to Mnesia as a Checkpoint.

2.5.2 State recovery

If the Leader crashes after the betting phase but before the round ends, no bets are lost.

1. A new Leader is elected via the Bully Algorithm.
2. The new Leader reads the latest Checkpoint from Mnesia.
3. The new Leader resumes the game, spins the wheel, calculates the outcomes based on the recovered ledger, and correctly issues payouts and refunds over RabbitMQ, ensuring zero financial loss for the players.

3. Implementation

3.1 Database Node

The database is implemented using a MySQL server and it is responsible for keeping all users accounts and bets information.

3.1.1 User table

User is one of the tables implemented in the database. It contains the following columns:

- **username:** username associated to the account.

- password: hashed password.
- balance: current wallet balance of the user.

3.1.2 Bet table

The Bet table tracks all wagers. It contains:

- bet_id: unique UUID for the bet.
- username: reference to the user.
- amount: wager amount.
- segment: the target segment of the wheel.
- status: PENDING, WON, LOST, REFUNDED.
- round: the round number it was placed in.

3.2 Central Node (Java)

The central node is the first point of contact for the clients with the distributed system. It consists in a Web Server implemented using the Springboot framework.

3.2.1 Source code structure

The source code is organized in the following packages:

- config: contains classes for handling the configuration parameters of the security and web sockets.
- controller: Springboot controllers for mapping all endpoints.
- dto: classes used for instantiating the Data Transfer Objects.
- entity: classes for representing the database data.
- rabbitmq: classes where the AMQP logic and listeners are developed.
- repository: classes for the database interaction.
- security: classes for interception and validation of requests.
- service: classes with the business logic.

3.2.2 The config package

It contains classes like SecurityConfig and WebSocketConfig.

- SecurityConfig: It defines Java Beans useful for the security aspects. The SecurityFilterChain bean defines a filter to be applied in incoming requests and protects endpoints.
- WebSocketConfig: Configures STOMP over WebSocket for real-time game updates.

3.2.3 The controller package

A total of 3 controllers are used.

- AuthController: Provides endpoints used for handling login and registration.
- WalletController: Provides REST API endpoints for requests relative to money and bets:
 - POST /api/wallet/place-bet: validates user balances, creates a unique bet_id, uses pessimistic locking (@Lock(LockModeType.PESSIMISTIC_WRITE)) on the database to prevent double-spending, and forwards the payload to RabbitMQ.
- GameController: Handles interactions during the mini-games (e.g., submitting a choice for CashHunt).

3.2.4 The rabbitmq package

Crucial for the asynchronous integration with the Game Engine.

- PayoutListener: Listens to the results_queue AMQP queue. When a round finishes, it securely credits winning bets to user wallets. It ensures idempotency by verifying the bet status, thus avoiding duplicate payouts.
- RefundListener: Listens to the refunds_queue. If the Erlang engine rejects a bet, this listener restores the deducted balance.
- GameStateCache: An atomic cache holding the most recent state broadcasted by the engine.

3.3 Erlang Game Engine

The Game Engine is designed to handle the core game logic and runs on a distributed Erlang cluster.

3.3.1 Source code structure

The following files are defined in the src/ directory:

- game_engine_app.src — configuration file of the application.
- game_engine_app.erl — application entry point.
- game_engine_sup.erl — node supervisor.
- cluster_manager.erl — manages the distributed cluster state.
- leader_election.erl — implements the Bully Algorithm.
- rabbitmq_manager.erl & worker.erl — AMQP handlers.
- wheel_process.erl — core game logic implemented via gen_server.
- cl_recorder.erl & snapshot.erl — Chandy-Lamport snapshot algorithm.
- minigames_sup.erl and mini-games modules (pachinko.erl, cashhunt.erl, etc.).

3.3.2 Fault Tolerance Modules

- leader_election.erl: Implements the fault-tolerance mechanism. It runs periodically, sending heartbeats to peer nodes. If it detects that the highest-ID node is dead, it promotes its own node if it holds the highest surviving ID, taking over the wheel_process duties.
- cluster_manager.erl: Maintains the lists of active nodes and configured nodes to evaluate quorums and avoid split-brain scenarios.

3.3.3 Core Game Modules

- wheel_process.erl:
 - Behaviour: gen_server.
 - Responsibilities: This process manages the timer lifecycle. When the betting phase ends, it coordinates with the snapshot modules to lock the bets. Then, it calculates the winning segment. If the segment is a number, it computes payouts. If it's a mini-game, it delegates the execution to the specific mini-game module.
- worker.erl: Dedicated processes that consume AMQP messages from bets_queue. In a Standby node, a worker simply routes the bet to the Leader node via standard Erlang Distribution (!). In the Leader node, the worker passes the bet to the local wheel_process.

- `cl_recorder.erl`: A pure functional module that records the local state of a worker when the "Snapshot Marker" is received, essential to build the globally consistent Checkpoint stored in Mnesia.

3.3.4 Mini-games

The mini-games are spawned as separate processes under `minigames_sup.erl`. For example, `cashhunt.erl` initializes a 9x12 grid of 108 hidden multipliers using a realistic probability distribution. It waits for choices routed from the Java Gateway and finally computes the exact payout for each participating user.

4. Demonstration

4.1 Run the system

To launch the project, you need multiple terminals. The startup sequence is as follows:

1. Start RabbitMQ (Terminal 1) Navigate to the RabbitMQ installation folder and start the server:

```
cd "C:\Program Files\RabbitMQ Server\rabbitmq_server-4.3.4\sbin"
.\rabbitmq-server.bat
(Management interface available at http://localhost:15672)
```

2. Start Java Gateway (Terminal 2) Navigate to the java-gateway folder and run the Spring Boot application:

```
cd java-gateway
mvn spring-boot:run
```

3. Start the Erlang Cluster (Terminals 3, 4, 5) Navigate to the Erlang game engine directory. You need to start 3 instances using the same cookie but different node names to form the cluster. (Note: Use `rebar3` shell as `erl` direct calls might fail to expand paths on Windows) Terminal 3 (Node 1):

```
cmd
cd erlang-engine\game_engine
set ERL_FLAGS=-sname game1@localhost -setcookie crazytime
escript ..\rebar3 shell
```

Terminal 4 (Node 2):

```
cd erlang-engine\game_engine
set ERL_FLAGS=-sname game2@localhost -setcookie crazytime
```

```
escript ../rebar3 shell
```

Terminal 5 (Node 3 - Leader):

```
cd erlang-engine\game_engine
set ERL_FLAGS=-sname game3@localhost -setcookie crazytime
escript ../rebar3 shell
```

Note for PowerShell users: set the environment variable using `$env:ERL_FLAGS = "-sname game1@localhost -setcookie crazytime"`.

Once the 3 nodes are up, the Bully Algorithm automatically elects game3 as the Leader (Dealer), while the other two become Standby nodes.

4.2 Interaction

You can interact with the system via the Web UI provided by the Java Gateway at <http://localhost:8080>, or via REST calls using curl:

```
# Register User
curl.exe -X POST
"http://localhost:8080/api/wallet/register?username=testuser"
# Place Bet
curl.exe -X POST
"http://localhost:8080/api/wallet/place-bet?username=testuser&amount=10.00&segment=5"
```